

Name: Hope L Chibondwe

Reg Number:H250404G

Course: Object Oriented Programming

Department: Computer Science

Lab report: 11 & Mini-project

LAB 11: Design Patterns and the Mini-Project

Objectives

1) Recognize four common patterns (Singleton, Factory, Strategy, Observer) and the problem each
   one solves.
2) Apply SOLID-style thinking: single responsibility, open for extension, program to interfaces.
3) Combine classes, interfaces, collections, exceptions and file I/O in one working application.

Sample programs
SingletonFactoryDemo.java

```
// ---------- SINGLETON ----------
class AppConfig {
private static final AppConfig INSTANCE = new AppConfig(); // created once
private final String institution = "Example University";
private AppConfig() { } // EXPLAIN: private constructor
// This prevents instantiation from outside the class
public static AppConfig getInstance() { return INSTANCE; }
public String getInstitution() { return institution; }
}
// ---------- FACTORY ----------
interface Notification {
void send(String message);
}
class EmailNotification implements Notification {
public void send(String m) { System.out.println("Email: " + m); }
}
class SmsNotification implements Notification {
public void send(String m) { System.out.println("SMS: " + m); }
}
class NotificationFactory {
// EXPLAIN: callers depend on the Notification interface, never on the concrete
classes

static Notification create(String type) {
return switch (type.toLowerCase()) {
case "email" -> new EmailNotification();
case "sms" -> new SmsNotification();
default -> throw new IllegalArgumentException("Unknown type: " + type);
};
}
```

```
}
public class SingletonFactoryDemo {
public static void main(String[] args) {
AppConfig a = AppConfig.getInstance();
AppConfig b = AppConfig.getInstance();
System.out.println("Same instance? " + (a == b) + " -> " +
a.getInstitution());
NotificationFactory.create("email").send("Lab 11 is open");
NotificationFactory.create("sms").send("Lab 11 is open");
}
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11> javac SingletonFactoryDemo.java
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11> java SingletonFactoryDemo
Same instance? true -> Example University
Email: Lab 11 is open
SMS: Lab 11 is open
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11>

StrategyObserverDemo.java

```
import java.util.ArrayList;
import java.util.List;
// ---------- STRATEGY ----------
interface DiscountStrategy {
double apply(double price);
}
class Checkout {
private DiscountStrategy strategy;
Checkout(DiscountStrategy strategy) { this.strategy = strategy; }
void setStrategy(DiscountStrategy s) { this.strategy = s; } // change behaviour
at run time

double total(double price) { return strategy.apply(price); }
}
// ---------- OBSERVER ----------
interface GradeListener {
void onGradePublished(String course, int mark);
}
class GradeBook {
```

```
private final List<GradeListener> listeners = new ArrayList<>();
void subscribe(GradeListener l) { listeners.add(l); }
void publish(String course, int mark) {
for (GradeListener l : listeners) l.onGradePublished(course, mark); // notify all

}
}
public class StrategyObserverDemo {
public static void main(String[] args) {
// Strategies are one-method interfaces, so lambdas work well
Checkout checkout = new Checkout(price -> price); // no discount
System.out.println("Regular: " + checkout.total(200));
checkout.setStrategy(price -> price * 0.90); // 10% student discount

System.out.println("Student: " + checkout.total(200));
GradeBook book = new GradeBook();
book.subscribe((course, mark) -> System.out.println("Student portal: " + course +
" = " + mark));

book.subscribe((course, mark) -> {
if (mark < 50) System.out.println("Advisor alert: " + course + "failed");

});
book.publish("OOP", 72);
book.publish("Databases", 45);
}
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11> javac StrategyObserverDemo.java
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11> java StrategyObserverDemo
Regular: 200.0
Student: 180.0
Student portal: OOP = 72
Student portal: Databases = 45
Advisor alert: Databasesfailed
PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\lab11>

Program 11.1: Singleton and Factory - Explanation

Singleton (AppConfig)

1) The constructor is private, so no other class can write new AppConfig().
2) The one instance is created inside the class and returned by getInstance().
3) The output Same instance? true shows that a and b are the same object.
4) Use it when exactly one shared object is needed, such as configuration or a logger.

Factory (NotificationFactory)

1) create("email") or create("sms") returns a Notification, and the caller never writes new EmailNotification().
2) The caller depends on the Notification interface, not on the concrete classes.
3) To add a new type, such as WhatsApp, you add one class and one case in the factory, and the caller's code does not change.
4) It shows polymorphism and abstraction. An unknown type throws an IllegalArgumentException.

Program 11.2: Strategy and Observer

Strategy (Checkout and DiscountStrategy)

1) DiscountStrategy is a one-method interface, so each discount rule can be written as a lambda.
2) Checkout holds the current strategy, and setStrategy changes it while the program runs.
3) Regular price: 200.0. After switching to the 10% student discount: 180.0. The Checkout class itself was not changed.
4) It shows composition over inheritance, because the behaviour is plugged in instead of inherited.

Observer (GradeBook and GradeListener)

1) GradeBook keeps a list of listeners, and publish tells every one of them.
2) The student portal prints every mark, and the advisor listener only reacts to a mark below 50.
3) That is why Databases = 45 produced the extra line Advisor alert: Databases failed.
4) It shows loose coupling, because GradeBook knows only the GradeListener interface and not who is listening.

Mini-Project

1) LibraryItem (abstract) with Book and Dvd: inheritance and abstraction. Each subclass has its own loanPeriodDays(), so the same call gives different results (polymorphism).
2) Member and Loan with private fields and validation: encapsulation.
3) Borrowable interface: a contract for borrow(Member) and giveBack(), so the library code depends on the interface.
4) ItemNotAvailableException and MemberLimitExceededException: custom checked exceptions, so callers must handle the case where an item is out or a member already has 3 loans.
5) Map<String, LibraryItem> and Map<String, Member>: items and members are looked up by id in constant time.
6) CSV save and load: data survives between runs, using Files.write and Files.readAllLines, with IOException handled.
7) Search by title with streams: filter, sorted and toList keep the code short and the source list unchanged.
8) The menu: a loop with a Scanner that calls the Library methods, so the user interface is separate from the logic.

Design choices

- Each class has one job (single responsibility), for example Library holds the rules and the menu class only handles input and output.
- You could also say whether you used a pattern, for example a Factory to create Book and Dvd from CSV lines, or a Singleton for the Library. Only mention one if you really used it.

Main.java

```
import java.io.IOException;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.List;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        LibraryStorage storage = new LibraryStorage(Path.of(args.length > 0 ?
args[0] : "data"));
        Library library;
```

```
        try {
            library = storage.load();
            System.out.println("Loaded " + library.getItems().size() + " items
and "
                    + library.getMembers().size() + " members.");
        } catch (IOException | RuntimeException e) {
            System.out.println("Could not load saved data (" + e.getMessage() +
"); starting empty.");
            library = new Library();
        }

        Scanner in = new Scanner(System.in);
        boolean running = true;
        while (running) {
            printMenu();
            String choice = in.hasNextLine() ? in.nextLine().trim() : "0";
            try {
                switch (choice) {
                    case "1" -> addItem(library, in);
                    case "2" -> registerMember(library, in);
                    case "3" -> borrow(library, in);
                    case "4" -> giveBack(library, in);
                    case "5" -> listOverdue(library);
                    case "6" -> search(library, in);
                    case "0" -> running = false;
                    default -> System.out.println("Unknown option.");
                }
            } catch (ItemNotAvailableException | MemberLimitExceededException e)
{
                System.out.println("Cannot borrow: " + e.getMessage());
            } catch (IllegalArgumentException | IllegalStateException e) {
                System.out.println("Error: " + e.getMessage());
            }
        }

        try {
            storage.save(library);
            System.out.println("Data saved. Goodbye!");
        } catch (IOException e) {
            System.out.println("Failed to save data: " + e.getMessage());
        }
    }

    private static void printMenu() {
        System.out.println("""
```

```
                === Library Menu ===
                1. Add item
                2. Register member
                3. Borrow item
                4. Return item
                5. List overdue items
                6. Search by title
                0. Save and exit""");
        System.out.print("> ");
    }

    private static String ask(Scanner in, String prompt) {
        System.out.print(prompt + ": ");
        return in.nextLine().trim();
    }

    private static void addItem(Library lib, Scanner in) {
        String type = ask(in, "Type (B = book, D = dvd)").toUpperCase();
        String id = ask(in, "Item id");
        String title = ask(in, "Title");
        switch (type) {
            case "B" -> lib.addItem(new Book(id, title, ask(in, "Author")));
            case "D" -> {
                String text = ask(in, "Runtime in minutes");
                try {
                    lib.addItem(new Dvd(id, title, Integer.parseInt(text)));
                } catch (NumberFormatException e) {
                    throw new IllegalArgumentException("Runtime must be a whole
number");
                }
            }
            default -> throw new IllegalArgumentException("Type must be B or D");
        }
        System.out.println("Item added.");
    }

    private static void registerMember(Library lib, Scanner in) {
        lib.registerMember(new Member(ask(in, "Member id"), ask(in, "Name")));
        System.out.println("Member registered.");
    }

    private static void borrow(Library lib, Scanner in)
            throws ItemNotAvailableException, MemberLimitExceededException {
        String memberId = ask(in, "Member id");
```

```
        String itemId = ask(in, "Item id");
        lib.borrow(memberId, itemId);
        System.out.println("Borrowed. Due " +
lib.findItem(itemId).getCurrentLoan().getDueDate());
    }

    private static void giveBack(Library lib, Scanner in) {
        lib.giveBack(ask(in, "Item id"));
        System.out.println("Item returned.");
    }

    private static void listOverdue(Library lib) {
        LocalDate today = LocalDate.now();
        List<Loan> overdue = lib.overdueLoans(today);
        if (overdue.isEmpty()) { System.out.println("No overdue items."); return;
}
        overdue.forEach(l -> System.out.println(l + " (" + l.daysOverdue(today) +
" days overdue)"));
    }

    private static void search(Library lib, Scanner in) {
        List<LibraryItem> results = lib.searchByTitle(ask(in, "Search text"));
        if (results.isEmpty()) System.out.println("No matches.");
        else results.forEach(System.out::println);
    }
}
```

Output

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\mini-project\library-project\library> javac *.java

PS C:\Users\hopec\OneDrive\Documents\java-oop-labs\mini-project\library-project\library> java Main

Loaded 0 items and 0 members.

=== Library Menu ===

1. Add item

2. Register member

3. Borrow item

4. Return item

5. List overdue items

6. Search by title

0. Save and exit

>

Conclusion

A design pattern is a named solution to a recurring design problem. Singleton controls how many instances exist, Factory hides which class is created, Strategy swaps behavior at run time, and Observer notifies many listeners without tight links. The mini-project combined encapsulation, inheritance, polymorphism, abstraction, exceptions, collections, streams and file handling in one program.